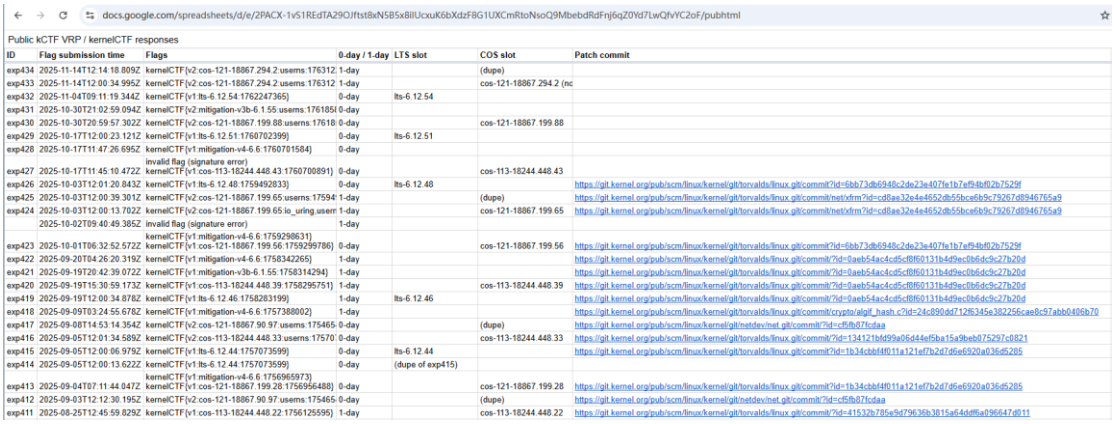
以往进行漏洞复现都是跟着一些师傅去学习，这是笔者第一次完全独立复现，所以写一篇记录一下

近期笔者从 360 的师傅口中得知一个有意思的链接

发现 10 月 3 号刚爆出一个有 commit 的 0day


docs.google.com/spreadsheets/d/e/2PACX-1vS1REdTA29OJftst8xN5B5x8iIUcxuK6bXdzF8G1UXCmRtoNsoQ9MbebdRdFnj6qZ0Yd7LwQfvYC2oF/pubhtml

Public kCTF VRP / kernelCTF responses

ID	Flag submission time	Flags	0-day / 1-day	LTS slot	COS slot	Patch commit
exp434	2025-11-14T12:14:18.809Z	kernelCTF{v2:cos-121-18867.294.2:usems:176312	1-day		(dupe)	
exp433	2025-11-14T12:00:34.995Z	kernelCTF{v2:cos-121-18867.294.2:usems:176312	1-day		cos-121-18867.294.2 (nc	
exp432	2025-11-04T09:11:19.344Z	kernelCTF{v1:lts-6.12.54:1762247365}	0-day	lts-6.12.54		
exp431	2025-10-30T21:02:59.094Z	kernelCTF{v2:mitigation-v3b-6.1.55:usems:176185(	0-day			
exp430	2025-10-30T20:59:57.302Z	kernelCTF{v2:cos-121-18867.199.88:usems:17618	0-day		cos-121-18867.199.88	
exp429	2025-10-17T12:00:23.121Z	kernelCTF{v1:lts-6.12.51:1760702399}	0-day	lts-6.12.51		
exp428	2025-10-17T11:47:26.695Z	kernelCTF{v1:mitigation-v4-6.6:1760701584}	0-day			
exp427	2025-10-17T11:45:10.472Z	invalid flag (signature error) kernelCTF{v1:cos-113-18244.448.43:1760700891}	0-day		cos-113-18244.448.43	
exp426	2025-10-03T12:01:20.843Z	kernelCTF{v1:lts-6.12.48:1759492833}	0-day	lts-6.12.48		https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=6bb73db6948c2de23a407fa1b7ef94bf02b729f
exp425	2025-10-03T12:00:39.301Z	kernelCTF{v2:cos-121-18867.199.65:usems:17594	1-day		(dupe)	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/net/xfrm?id=cd8ae32e4e4652db55bce6b9c79267d8946765a9
exp424	2025-10-03T12:00:13.702Z	kernelCTF{v2:cos-121-18867.199.65:io_uring,usern	1-day		cos-121-18867.199.65	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/net/xfrm?id=cd8ae32e4e4652db55bce6b9c79267d8946765a9
	2025-10-02T09:40:49.385Z	invalid flag (signature error)	1-day			
exp423	2025-10-01T06:32:52.572Z	kernelCTF{v1:mitigation-v4-6.6:1759298631} kernelCTF{v1:cos-121-18867.199.56:1759299786}	0-day		cos-121-18867.199.56	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=6bb73db6948c2de23a407fa1b7ef94bf02b729f
exp422	2025-09-20T04:26:20.319Z	kernelCTF{v1:mitigation-v4-6.6:1758342265}	1-day			https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=0aeb54ac4cd5cf0f60131b4d9ec0b6dc9c27b20d
exp421	2025-09-19T20:42:39.072Z	kernelCTF{v1:mitigation-v3b-6.1.55:1758314294}	1-day			https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=0aeb54ac4cd5cf0f60131b4d9ec0b6dc9c27b20d
exp420	2025-09-19T15:30:59.173Z	kernelCTF{v1:cos-113-18244.448.39:1758295751}	1-day		cos-113-18244.448.39	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=0aeb54ac4cd5cf0f60131b4d9ec0b6dc9c27b20d
exp419	2025-09-19T12:00:34.878Z	kernelCTF{v1:lts-6.12.46:1758283199}	1-day	lts-6.12.46		https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=0aeb54ac4cd5cf0f60131b4d9ec0b6dc9c27b20d
exp418	2025-09-09T03:24:55.678Z	kernelCTF{v1:mitigation-v4-6.6:1757388002}	1-day			https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/crypto/algif_hash.c?id=24c890dd712f6345e382256cae8c97abb0406b70
exp417	2025-09-08T14:53:14.354Z	kernelCTF{v2:cos-121-18867.90.97:usems:175465	0-day		(dupe)	https://git.kernel.org/pub/scm/linux/kernel/git/netdev/net.git/commit/?id=cf5fb87fcdaa
exp416	2025-09-05T12:01:34.589Z	kernelCTF{v2:cos-113-18244.448.33:usems:17570	0-day		cos-113-18244.448.33	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=134121bfd99a06d44ef5ba15a9beb075297c0821
exp415	2025-09-05T12:00:06.979Z	kernelCTF{v1:lts-6.12.44:1757073599}	0-day	lts-6.12.44		https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=1b34cbbf4f011a121ef7b2d7d6e6920a036d5285
exp414	2025-09-05T12:00:13.622Z	kernelCTF{v1:lts-6.12.44:1757073599}	0-day	(dupe of exp415)		
exp413	2025-09-04T07:11:44.047Z	kernelCTF{v1:mitigation-v4-6.6:1756965973} kernelCTF{v1:cos-121-18867.199.28:1756956488}	0-day		cos-121-18867.199.28	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=1b34cbbf4f011a121ef7b2d7d6e6920a036d5285
exp412	2025-09-03T12:12:30.195Z	kernelCTF{v2:cos-121-18867.90.97:usems:175465	0-day		(dupe)	https://git.kernel.org/pub/scm/linux/kernel/git/netdev/net.git/commit/?id=cf5fb87fcdaa
exp411	2025-08-25T12:45:59.829Z	kernelCTF{v1:cos-113-18244.448.22:1756125595}	1-day		cos-113-18244.448.22	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/commit/?id=41532b785e9d79636b3815a64ddf6a096647d011

查看该 commit 发现是加密子系统相关

commit 6bb73db6948c2de23e407fe1b7ef94bf02b7529f (patch)
tree e0734b87f421b9e1387c0427d5fc5b56bc73f1a0
parent 229c586b5e86979badb7cb0d38717b88a9e95ddd (diff)
download linux-6bb73db6948c2de23e407fe1b7ef94bf02b7529f.tar.gz

crypto: essiv - Check ssize for decryption and in-place encryption

```
Move the ssize check to the start in essiv_aead_crypt so that
it's also checked for decryption and in-place encryption.

Reported-by: Muhammad Alifa Ramdhan <ramdhan@starlabs.sg>
Fixes: be1eb7f78aa8 ("crypto: essiv - create wrapper template for ESSIV generation")
Signed-off-by: Herbert Xu <herbert@gondor.apana.org.au>
```

Diffstat

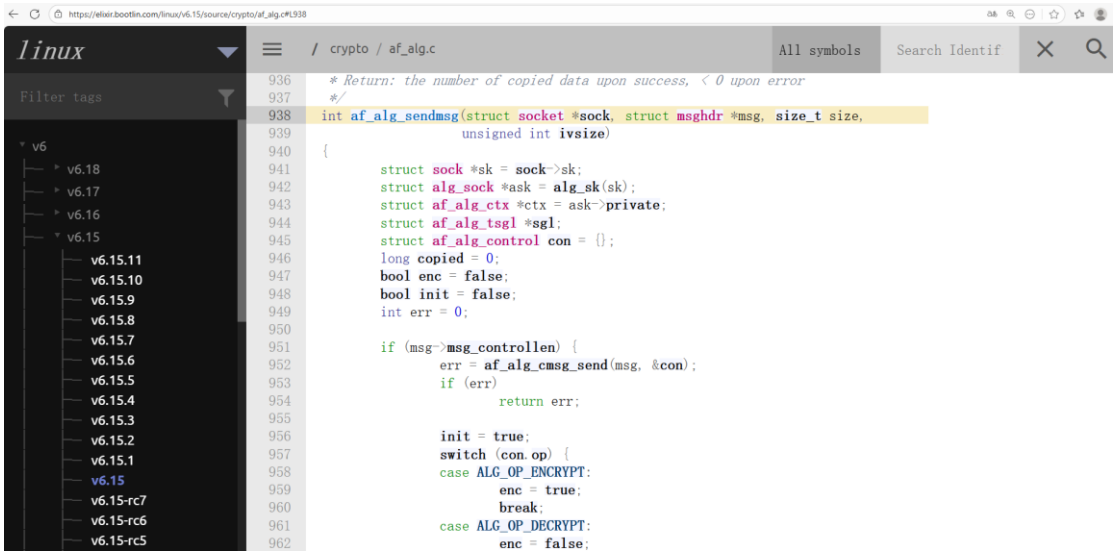
-rw-r--r-- crypto/essiv.c 14

1 files changed, 6 insertions, 8 deletions

```
diff --git a/crypto/essiv.c b/crypto/essiv.c
index d003b78fcd855a..a47a3eab693519 100644
--- a/crypto/essiv.c
+++ b/crypto/essiv.c
@@ -186,9 +186,14 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
        const struct essiv_tfm_ctx *tctx = crypto_aead_ctx(tfm);
        struct essiv_aead_request_ctx *rctx = aead_request_ctx(req);
        struct aead_request *subreq = &rctx->aead_req;
+       int ivsize = crypto_aead_ivsize(tfm);
+       int ssize = req->assoclen - ivsize;
        struct scatterlist *src = req->src;
        int err;

+       if (ssize < 0)
+               return -EINVAL;
+
        crypto_cipher_encrypt_one(tctx->essiv_cipher, req->iv, req->iv);

        /*
@@ -198,19 +203,12 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
         */
        rctx->assoc = NULL;
        if (req->src == req->dst || !enc) {
-               scatterwalk_map_and_copy(req->iv, req->dst,
-                                        req->assoclen - crypto_aead_ivsize(tfm),
-                                        crypto_aead_ivsize(tfm), 1);
+               scatterwalk_map_and_copy(req->iv, req->dst, ssize, ivsize, 1);
        } else {
                u8 *iv = (u8 *)aead_request_ctx(req) + tctx->ivoffset;
-               int ivsize = crypto_aead_ivsize(tfm);
-               int ssize = req->assoclen - ivsize;
                struct scatterlist *sg;
                int nents;

-               if (ssize < 0)
-                       return -EINVAL;
-
```

在此之前笔者对 Linux 加密子系统一无所知，所幸刚好看了 360 的师傅复现 AF_ALG 那篇文章，翻过一些源码有了基础的了解

本文源码基于 Linux6.15
AF_ALG 是 Linux 内核通过 socket 提供加解密的用户态接口,主要支持四种类型，对应四个文件或者说内核模块：
algif_hash -> 处理生成哈希值
algif_rng -> 处理生成随机数
algif_skcipher -> 处理对称加密
algif_aead -> 处理认证加密
af_alg_sendmsg 是处理 af_alg 发送请求的函数，主要分发加解密操作，但由于生成哈希值随机数并不涉及到加解密，所以只有 algif_skcipher 和 algif_aead 调用



内核的 sendmsg 相关函数是处理发送请求，recvmsg 时才会进行具体的加解密分发，比如 aead 类型的加解密处理最终调用 aead_recvmsg 完成

```
325 static int aead_recvmsg(struct socket *sock, struct msghdr *msg,
326                         size_t ignored, int flags)
327 {
328         struct sock *sk = sock->sk;
329         int ret = 0;
330
331         lock_sock(sk);
332         while (msg_data_left(msg)) {
333                 int err = _aead_recvmsg(sock, msg, ignored, flags);
334
335                 /*
336                  * This error covers -EIOCBQUEUED which implies that we can
337                  * only handle one AIO request. If the caller wants to have
338                  * multiple AIO requests in parallel, he must make multiple
339                  * separate AIO calls.
340                  *
341                  * Also return the error if no data has been processed so far.
342                  */
343                 if (err <= 0) {
344                         if (err == -EIOCBQUEUED || err == -EBADMSG || !ret)
345                                 ret = err;
346                         goto out;
347                 }
```

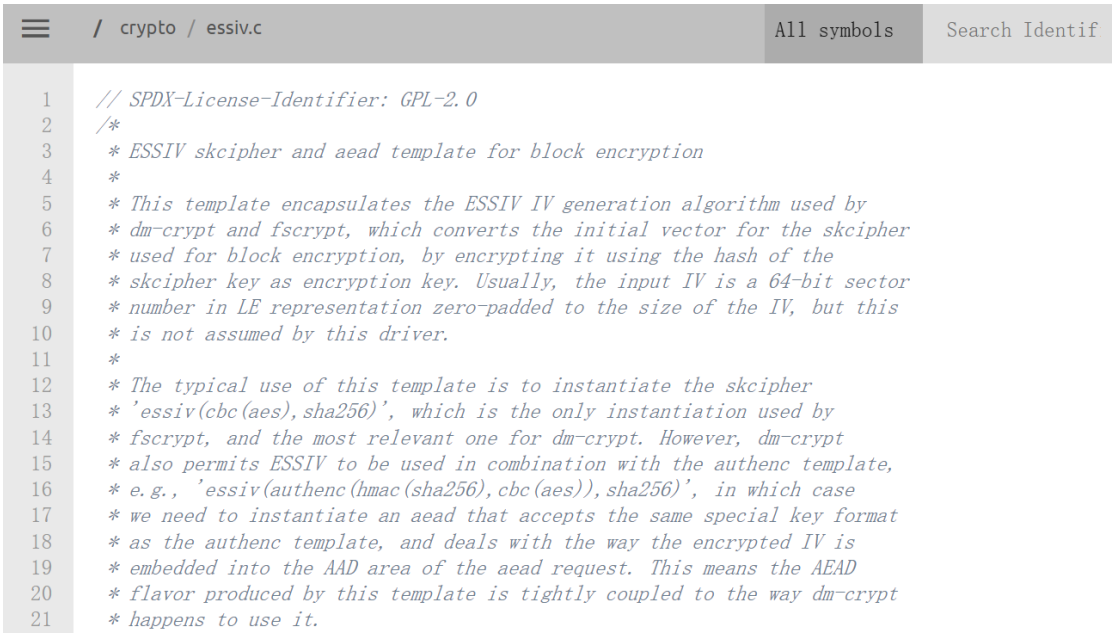
_aead_recvmsg 最后会分发到相应的加解密函数进行处理

```
                aead_request_set_callback(&areq->cra_u.aead_req,
                                          CRYPTO_TFM_REQ_MAY_SLEEP,
                                          af_alg_async_cb, areq);
                err = ctx->enc ? crypto_aead_encrypt(&areq->cra_u.aead_req) :
                                 crypto_aead_decrypt(&areq->cra_u.aead_req);

                /* AIO operation in progress */
                if (err == -EINPROGRESS)
                        return -EIOCBQUEUED;

                sock_put(sk);
        } else {
                /* Synchronous operation */
                aead_request_set_callback(&areq->cra_u.aead_req,
                                          CRYPTO_TFM_REQ_MAY_SLEEP |
                                          CRYPTO_TFM_REQ_MAY_BACKLOG,
                                          crypto_req_done, &ctx->wait);
                err = crypto_wait_req(ctx->enc ?
                                crypto_aead_encrypt(&areq->cra_u.aead_req) :
                                crypto_aead_decrypt(&areq->cra_u.aead_req),
                                &ctx->wait);
```

而 patch 的 commit 主要 fix 的就是 essiv_aead_crypt 这个函数，显而易见其与 aead 加密相关，相关代码位于 crypto/essiv.c 这个文件

```
index d003b78fcd85..a47a3eab6935 100644
--- a/crypto/essiv.c
+++ b/crypto/essiv.c
@@ -186,9 +186,14 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
        const struct essiv_tfm_ctx *tctx = crypto_aead_ctx(tfm);
        struct essiv_aead_request_ctx *rctx = aead_request_ctx(req);
        struct aead_request *subreq = &rctx->aead_req;
+       int ivsize = crypto_aead_ivsize(tfm);
+       int ssize = req->assoclen - ivsize;
        struct scatterlist *src = req->src;
        int err;

+       if (ssize < 0)
+               return -EINVAL;
+
        crypto_cipher_encrypt_one(tctx->essiv_cipher, req->iv, req->iv);

        /*
@@ -198,19 +203,12 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
         */
        rctx->assoc = NULL;
        if (req->src == req->dst || !enc) {
-               scatterwalk_map_and_copy(req->iv, req->dst,
-                                        req->assoclen - crypto_aead_ivsize(tfm),
-                                        crypto_aead_ivsize(tfm), 1);
+               scatterwalk_map_and_copy(req->iv, req->dst, ssize, ivsize, 1);
        } else {
                u8 *iv = (u8 *)aead_request_ctx(req) + tctx->ivoffset;
-               int ivsize = crypto_aead_ivsize(tfm);
-               int ssize = req->assoclen - ivsize;
                struct scatterlist *sg;
                int nents;

-               if (ssize < 0)
-                       return -EINVAL;
-
:
```

分析该文件头部的 desc，大概就是对称加密用 essiv(cbc(aes),sha256)，认证加密用 essiv(authenc(hmac(sha256),cbc(aes)),sha256)，笔者对具体支持的加密算法不甚了解，直接就用它给的样板了



所以在用户态程序中则有如下定义：

```
struct sockaddr_alg sa = {
    .salg_family = AF_ALG,
    .salg_type = "aead",
    .salg_name = "essiv(authenc(hmac(sha256),cbc(aes)),sha256)"
};
```

后面的交互代码是调教 AI 出来的，跑出问题了调试内核找到报错的代码段反馈给 AI 让它进行修改，缝缝补补最终出来了这个勉强可用的 POC
最终出现了一个空指针解引用

```
test@syzkaller:/$ id
uid=1001(test) gid=1001(test) groups=1001(test)
test@syzkaller:/$ /poc
[ 4103.608680] BUG: kernel NULL pointer dereference, address: 000000000000000c
[ 4103.609047] #PF: supervisor read access in kernel mode
[ 4103.609263] #PF: error_code(0x0000) - not-present page
[ 4103.609365] PGD 108cc3067 P4D 108cc3067 PUD 106a43067 PMD 0
[ 4103.609365] Oops: Oops: 0000 [#7] SMP NOPTI
[ 4103.609365] CPU: 1 UID: 1001 PID: 601 Comm: poc Tainted: G      D            6.15.0 #4 PREEMPT(voluntary)
[ 4103.609365] Tainted: [D]=DIE
[ 4103.609365] Hardware name: QEMU Ubuntu 24.04 PC (i440FX + PIIX, 1996), BIOS 1.16.3-debian-1.16.3-2 04/01/2014
[ 4103.609365] RIP: 0010:memcpy_to_sglist+0x55/0xb0
[ 4103.609365] Code: d0 00 00 00 00 48 c7 45 d8 00 00 00 00 85 c9 74 39 8b 47 0c 89 f3 49 89 d5 41 89 cc 39 f0 73 11 29 c3 e8 8e 25 0c 00 48 89 c7
[ 4103.609365] RSP: 0018:ffffc90000b6b7f0 EFLAGS: 00000246
[ 4103.609365] RAX: 0000000000000000 RBX: 00000000fffffffc0 RCX: 0000000000000010
[ 4103.609365] RDX: 0000000000000000 RSI: 00000000fffffff0 RDI: 0000000000000000
[ 4103.609365] RBP: ffffc90000b6b828 R08: 0000000000000000 R09: 0000000000000000
[ 4103.609365] R10: 0000000000000000 R11: 0000000000000000 R12: 0000000000000010
[ 4103.609365] R13: ffff8881021646a0 R14: ffff8881013e21e8 R15: ffff888100aa3290
[ 4103.609365] FS:  0000000017df9380(0000) GS:ffff888235cde000(0000) knlGS:0000000000000000
[ 4103.609365] CS:  0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[ 4103.609365] CR2: 000000000000000c CR3: 0000000108d21000 CR4: 00000000000006f0
[ 4103.609365] Call Trace:
[ 4103.609365]  <TASK>
[ 4103.609365]  essiv_aead_crypt+0x1a6/0x330
[ 4103.609365]  essiv_aead_encrypt+0x13/0x20
[ 4103.609365]  crypto_aead_encrypt+0x20/0x40
[ 4103.609365]  aead_recvmsg+0x5f5/0x6c0
```

结合 desc 仔细分析这个 commit，不难看出看似增删了不少代码，但实则只做了两个操作：

1、 直接将 ivsize、ssize 都存进一个变量
2、 提前 check 了 ssize

```
@@ -186,9 +186,14 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
        const struct essiv_tfm_ctx *tctx = crypto_aead_ctx(tfm);
        struct essiv_aead_request_ctx *rctx = aead_request_ctx(req);
        struct aead_request *subreq = &rctx->aead_req;
+       int ivsize = crypto_aead_ivsize(tfm);
+       int ssize = req->assoclen - ivsize;
        struct scatterlist *src = req->src;
        int err;

+       if (ssize < 0)
+               return -EINVAL;
+
        crypto_cipher_encrypt_one(tctx->essiv_cipher, req->iv, req->iv);

        /*
@@ -198,19 +203,12 @@ static int essiv_aead_crypt(struct aead_request *req, bool enc)
         */
        rctx->assoc = NULL;
        if (req->src == req->dst || !enc) {
-               scatterwalk_map_and_copy(req->iv, req->dst,
-                                        req->assoclen - crypto_aead_ivsize(tfm),
-                                        crypto_aead_ivsize(tfm), 1);
+               scatterwalk_map_and_copy(req->iv, req->dst, ssize, ivsize, 1);
        } else {
                u8 *iv = (u8 *)aead_request_ctx(req) + tctx->ivoffset;
-               int ivsize = crypto_aead_ivsize(tfm);
-               int ssize = req->assoclen - ivsize;
                struct scatterlist *sg;
                int nents;

-               if (ssize < 0)
-                       return -EINVAL;
```

而这么修改的最终目的只有一个：使得 scatterwalk_map_and_copy 的第三个参数不小于 0

linux内核中scatterwalk_map_and_copy的函数原型是什么

在 Linux 内核中，`scatterwalk_map_and_copy` 函数的原型如下：

c 运行

```c
#include <linux/scatterlist.h>

void scatterwalk_map_and_copy(void *buf, struct scatterlist *sg,
                              unsigned int start, unsigned int len,
                              int out);
```

参数说明：

- `buf`：指向用户空间或内核空间的缓冲区（数据将被复制到这里，或从这里复制出去）。
- `sg`：指向 scatterlist 结构链表，描述了分散的内存区域。
- `start`：在 scatterlist 数据区域中的起始偏移量（从该位置开始复制）。
- `len`：要复制的数据长度。
- `out`：方向标志（`1` 表示从 `buf` 复制到 scatterlist；`0` 表示从 scatterlist 复制到 `buf`）。

第三个函数是起始偏移量，不难想到假使该值为负数会导致前向越界？
继续往下分析到 scatterwalk_start_at_pos 函数，前面我们传入的负数就是这个 pos，而由于比较对象是无符号整数被当成了大的整数进入了循环

```
static inline void scatterwalk_start_at_pos(struct scatter_walk *walk,
                                            struct scatterlist *sg,
                                            unsigned int pos)
{
        while (pos > sg->length) {
                pos -= sg->length;
                sg = sg_next(sg);
        }
        walk->sg = sg;
        walk->offset = sg->offset + pos;
}
```

POC 触发了第一个 check 直接返回了 NULL，而在循环条件中 sg->length 进行了解引用，最终导致了空指针解引用

```
26  struct scatterlist *sg_next(struct scatterlist *sg)
27  {
28          if (sg_is_last(sg))
29                  return NULL;
30
31          sg++;
32          if (unlikely(sg_is_chain(sg)))
33                  sg = sg_chain_ptr(sg);
34
35          return sg;
36  }
```

ps：ssize 的值是 aad 的长度减去 iv 的长度，aad 的长度是在 sendmsg 中通过 cmsg 控制消息传入的，笔者直接没有传入该值所以是 0，而 iv 的长度是 0x10 最终得出的 ssize 是-0x10 即 0xFFFFFFF0

/ crypto / af_alg.c　　All symbols　　Search Identif

```
975                         err = -EINVAL;
976                         goto unlock;
977                 }
978
979                 pr_info_once(
980                         "%s sent an empty control message without MSG_MORE.\n",
981                         current->comm);
982         }
983         ctx->init = true;
984
985         if (init) {
986                 ctx->enc = enc;
987                 if (con.iv)
988                         memcpy(ctx->iv, con.iv->iv, ivsize);
989
990                 ctx->aead_assoclen = con.aead_assoclen;
991         }
```

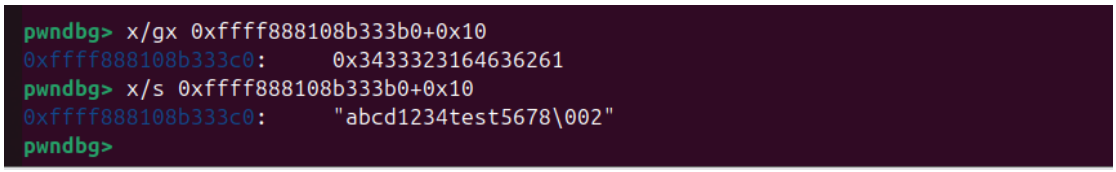
根据笔者目前的认知，好似空指针能造成提权的情况早已被修复？但出于对 kernelctf 的不了解，就默认以提权为成功标准去判断了，这让笔者百思不得其解利用这个漏洞的师傅是怎么完成提权的，最终只能提出一些大胆猜想。

正如前面所说起始偏移量为负数最终可能会导致前向越界，现在进行调试，首先笔者在 0xffffffff81a9cdc7 处（对应 while (pos > sg->length)这条代码）下断点

修改 esi（即 pos 这个变量）的值为 0，不进入循环

然后下断点在 0xffffffff81a9cd0e 处（内核解析完内存映射后进行实际写入操作的地方，对应源码中 memcpy(walk->addr, buf, to_copy)这条代码）
可以看到证实发生了前向越界写

正常情况下应该是写入到 aad 数据块的



最后笔者总结一下目前碰到的问题：

1、 对 scatterlist 不甚了解，发生漏洞的内存环境难以得知，因为笔者查看了一下漏洞内存前面存储的数据都是一些用户态指针，不知该堆喷什么去利用。并且写入的数据是生成的加密 essiv，也是半个不可控因素，想要利用只能考虑覆盖目标结构体的 size 成员转成越界任意读写原语？

```
pwndbg> x/4gx 0xffff888108b333b0-0xb0
0xffff888108b33300:     0x0000000000000000      0x0000000000000000
0xffff888108b33310:     0x0000000000000000      0x0000000000000000
pwndbg>
0xffff888108b33320:     0x0000000000000000      0x0000000000000000
0xffff888108b33330:     0x0000000000000000      0x0000000000000000
pwndbg>
0xffff888108b33340:     0x0000000000000000      0x0000000000000000
0xffff888108b33350:     0x0000000000000000      0x0000000000000000
pwndbg>
0xffff888108b33360:     0x0000000000000000      0x0000000000415c4c
0xffff888108b33370:     0x0000000000000000      0x0000000000000000
pwndbg>
0xffff888108b33380:     0x0000000000000000      0x0000000000000000
0xffff888108b33390:     0x0000000000000000      0x0000000000000000
pwndbg>
0xffff888108b333a0:     0x0000000000000038      0x0000000000000010
0xffff888108b333b0:     0x00007ffdb6491650      0x0000000000402233
pwndbg>
0xffff888108b333c0:     0x3433323164636261      0x3837363574736574
0xffff888108b333d0:     0x0000000000000002      0x00007ffdb6491778
pwndbg>
```

2、 上述操作仍均为设想，想落地实现需绕过这个循环条件的 check（利用 race condition？但一些未证实的信息说不能这么玩）

```
static inline void scatterwalk_start_at_pos(struct scatter_walk *walk,
                                            struct scatterlist *sg,
                                            unsigned int pos)
{
        while (pos > sg->length) {
                pos -= sg->length;
                sg = sg_next(sg);
        }
        walk->sg = sg;
        walk->offset = sg->offset + pos;
}
```

笔者能力有限，有些地方或许分析的不到位，对于一些知识的认知也有些片面，最终也没能成功弄出 exploit。只能抛砖引玉，欢迎各位师傅指正

Email：3156063554@qq.com